

Milestone 5 Report

Menna Wahba

Ahd Elsharkawy
Tarek Sherif

Rahma Ahmed
Youssef Amro

Omar Magdy

December 10, 2025

1 Literature Review

1.1 History of Robotic Arms

A robot is a reprogrammable, multifunctional manipulator designed to move material, parts, tools or specialized devices through variable programmed motions for the performance of a variety of tasks. Computer science, mechanical engineering, electrical engineering and other technologies are used to develop robots that can substitute for humans. Robots are widely used in manufacturing, assembly, packaging, mining, transport, earth and space exploration, surgery, weaponry, laboratory research, safety, and the mass production of consumer and industrial goods [1].

Robots can take on any form but some are made to resemble humans in appearance. This is said to help with the acceptance of a robot in certain replicative behaviors usually performed by people. Such robots attempt to replicate walking, lifting, speech, cognition, and basically anything a human can do. Many of today's robots are inspired by nature, contributing to the field of bio-inspired robotics. However, some people may alternatively perceive robots as dangerous technological ventures that will someday lead to the demise of the human race, either by outsmarting humans or out muscling us and taking over the world, or by turning us into completely technology-dependent beings who passively sit by and program robots to do all of our work [2].

The history of robotic arms dates back to the invention of the first industrial robotic arm, the Unimate, created by George Devol and marketed by Joseph Engelberger in 1961. This pioneering robotic arm featured hydraulic actuators and three axes, primarily used in industrial tasks such as material handling and welding, marking the beginning of automation in manufacturing [3, 4]. The evolution of robotic arms progressed through multiple generations incorporating servo-control, computer interfaces, and greater degrees of freedom, exemplified by innovations such as the Stanford arm developed in 1969, which featured six axes and sensor-based control [3, 4].

Beyond industrial applications, the technological advancement of robotic arms has extended into surgical and prosthetic fields. The da Vinci Surgical System, a notable example in medical robotics, integrates robotic arms to perform minimally invasive surgeries with high precision, expanding the arm's role from manufacturing to healthcare [3, 5]. More recent developments emphasize the integration of artificial intelligence and embodied intelligence in robotic arms, enabling adaptive, autonomous operation in complex environments [6].

Thus, robotic arms have evolved from simple hydraulically driven devices to sophisticated multifunctional robots with applications spanning industry, medicine, and beyond, continually advancing through innovations in control, sensing, and AI technologies [3, 4, 6].

1.2 Industrial Applications of Robotic Arms

Industrial robotic arms are programmable manipulators designed to replace humans in performing repetitive, dangerous, or precision-required tasks across various industries including automotive, electronics, and hazardous material handling. A robotic arm consists of links connected by joints that allow rotational or translational motion, with an end effector analogous to a human hand that can be designed for specific tasks like welding, gripping, or spinning. Modern control approaches utilize haptic technology and accelerometer sensors to capture natural human arm movements, allowing the processing unit to generate corresponding control signals that replicate these movements in the robotic arm, providing greater flexibility compared to traditional manual controllers where each joint is controlled separately [7].

Software simulation and control development employ platforms like LabVIEW for creating graphical user interfaces and Arduino for inverse kinematics calculations that convert desired end-effector positions into specific joint angles. The simulation process enables users to input object coordinates, visualize robot movements, and test pick-and-place operations in a virtual environment before physical implementation. Key design parameters for simulation include degrees of freedom (typically 3-6 DOF), workspace analysis, position accuracy, and bilateral communication between master control devices and slave robot systems with force feedback capabilities, ensuring effective performance validation prior to hardware deployment [7].

1.3 Laser Cutting

The use of these simulations and development helps robotic arms excel in environments where accuracy is critical. One of these interesting areas is laser cutting. Robotic laser cutting has strong potential to change the way metal manufacturing operates in the Industry 4.0 era. A study done by Nikose et al. (2024) compares traditional flat-bed CNC laser cutting machines with robotic laser cutting systems, focusing on flexibility, cost, accuracy, and digital integration. They note that while flat-bed machines remain the most common, robotic systems provide more adaptability, easier digital control, and better compatibility with automation and artificial intelligence. The authors conclude that robotic laser cutting can be a more efficient and affordable option for small and medium-sized manufacturers, especially those producing various products in small batches, and they recommend further research to improve and expand its industrial use [8].

Inspired by the research paper, the project presents a laser-cut robotic arm that can cut acrylic sheets using a small laser fixed at the end of the arm as the end-effector. The goal is to let the arm make clean and accurate cuts on given paths instead of doing it by hand to automate the whole process. Such systems are usually found in factories to make custom parts or signs. The design and testing will be done using ROS2, Python, and MuJoCo.

2 DH Convention and Kinematics

For our application we chose the 6 DOF **UR5E** robot for its light payload and it being suitable for applications that require a small-sized robot with sufficient reach and ability to perform precise, meticulous tasks [9]. Also due to the widely available datasets for it on the internet.

2.1 DH Convention

2.2 X-Z Frames

First we started by drawing X-Z frames over 2 different views as follows :

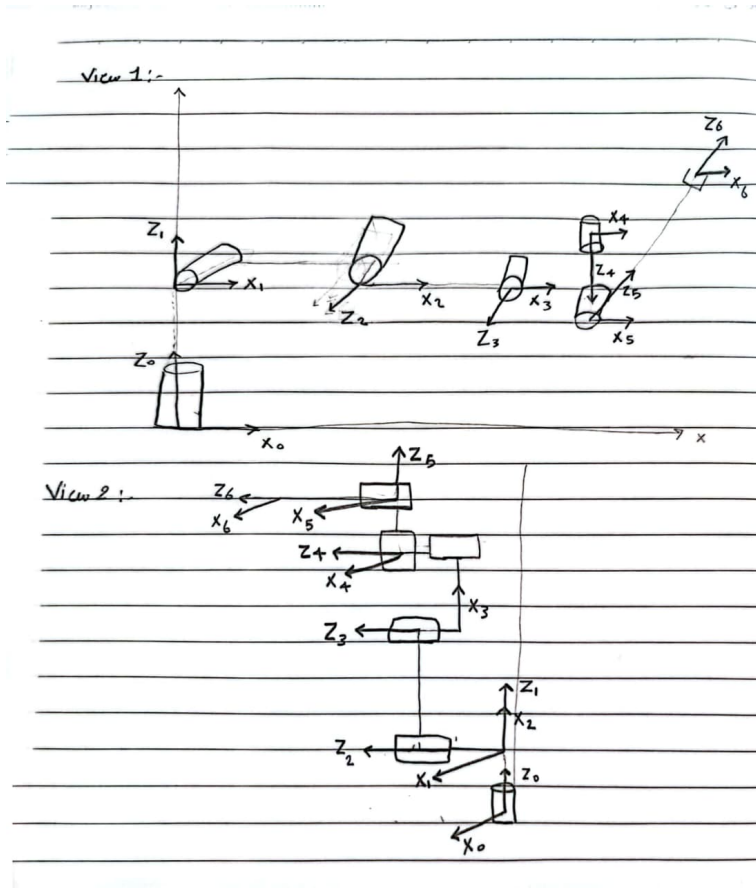


Figure 1: X-Z frames over 2 different views

2.2.1 DH-parameters Table

And then proceeded to calculate DH table values that turned out as follows:

Joint i	q_i	d_i	a_i	α_i [rad]
1	q_1	L1	0	$+\frac{\pi}{2}$
2	q_2	0	L2	0
3	q_3	0	L3	0
4	q_4	L4	0	$+\frac{\pi}{2}$
5	q_5	L5	0	$-\frac{\pi}{2}$
6	q_6	L6	0	0

2.2.2 Forward Kinematics

We then used the table to calculate forward kinematics. For our forward kinematics values we noticed that the x and y are reversed from the frames we used for table parameters and therefore multiplied them by -1. We tried multiple angles, 3 examples of which are :

1. no rotation

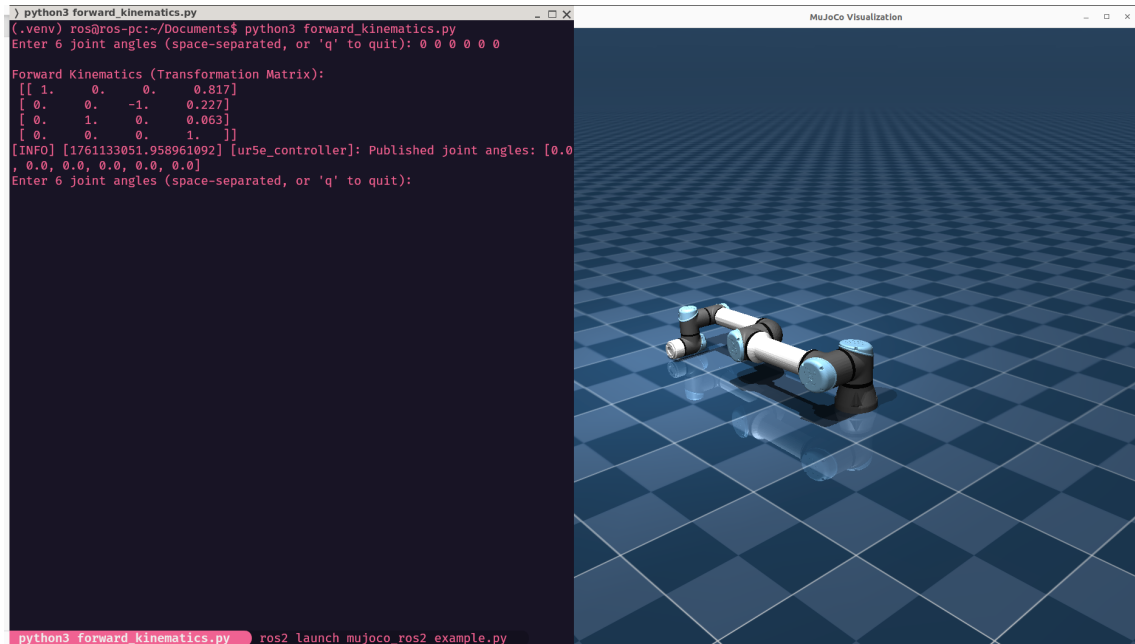


Figure 2: no rotation

2. -90° in x

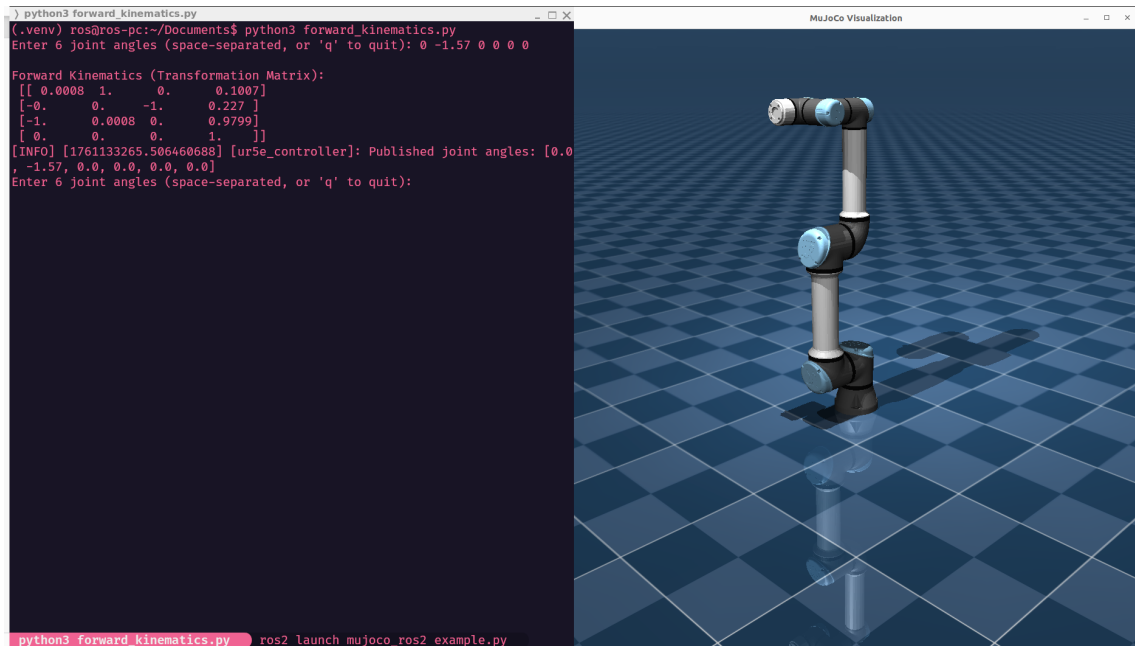


Figure 3: -90° rotation in x

3. 90° deg in z

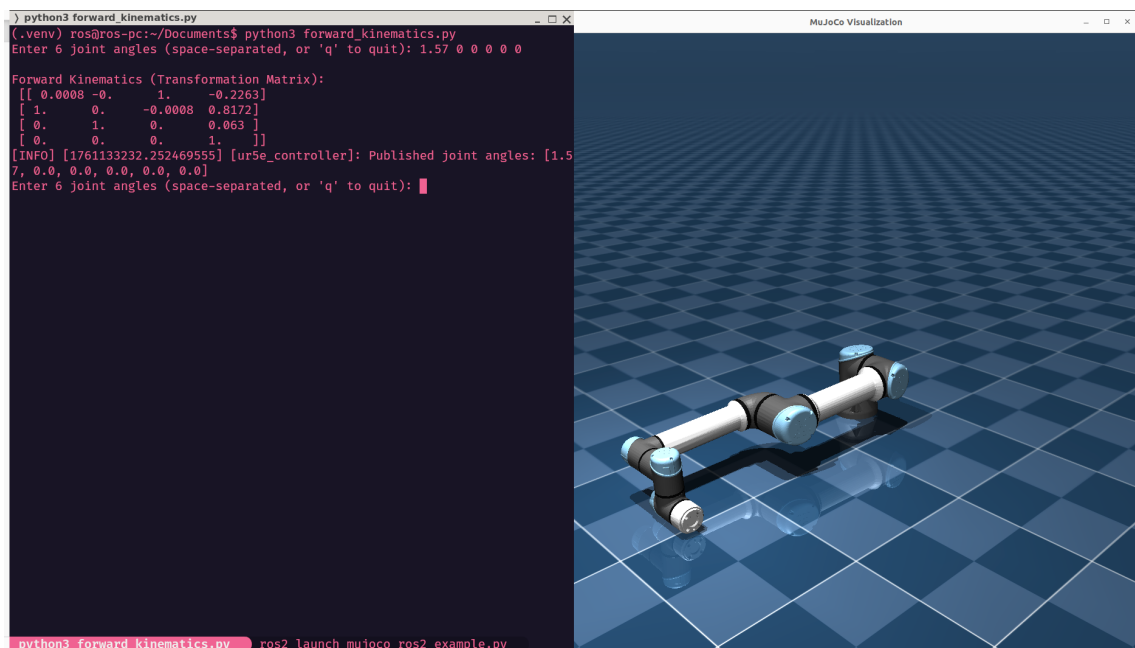


Figure 4: 90° rotation in z

We were then able to calculate the following table given that the robot part length were as follows :

$$L_{1-6} = [0.163, -0.425, -0.392, 0.127, 0.1, 0.1]$$

End-Effector Position	End-Effector Forward Position Equations
$x_{e.e}$	$0.1 \sin(q_1) \cos(q_5) + 0.127 \sin(q_1) - 0.1 \sin(q_5) \cos(q_1) \cos(q_2 + q_3 + q_4) + 0.1 \sin(q_2 + q_3 + q_4) \cos(q_1) - 0.425 \cos(q_1) \cos(q_2) - 0.392 \cos(q_1) \cos(q_2 + q_3)$
$y_{e.e}$	$-0.1 \sin(q_1) \sin(q_5) \cos(q_2 + q_3 + q_4) + 0.1 \sin(q_1) \sin(q_2 + q_3 + q_4) - 0.425 \sin(q_1) \cos(q_2) - 0.392 \sin(q_1) \cos(q_2 + q_3) - 0.1 \cos(q_1) \cos(q_5) - 0.127 \cos(q_1)$
$z_{e.e}$	$-0.425 \sin(q_2) - 0.1 \sin(q_5) \sin(q_2 + q_3 + q_4) - 0.392 \sin(q_2 + q_3) - 0.1 \cos(q_2 + q_3 + q_4) + 0.163$

Table 1: Forward kinematic solution for the UR5E robot

2.3 Inverse Kinematics

The python library ikpy.chain was used to calculate the inverse kinematics angles for each joint as follows:

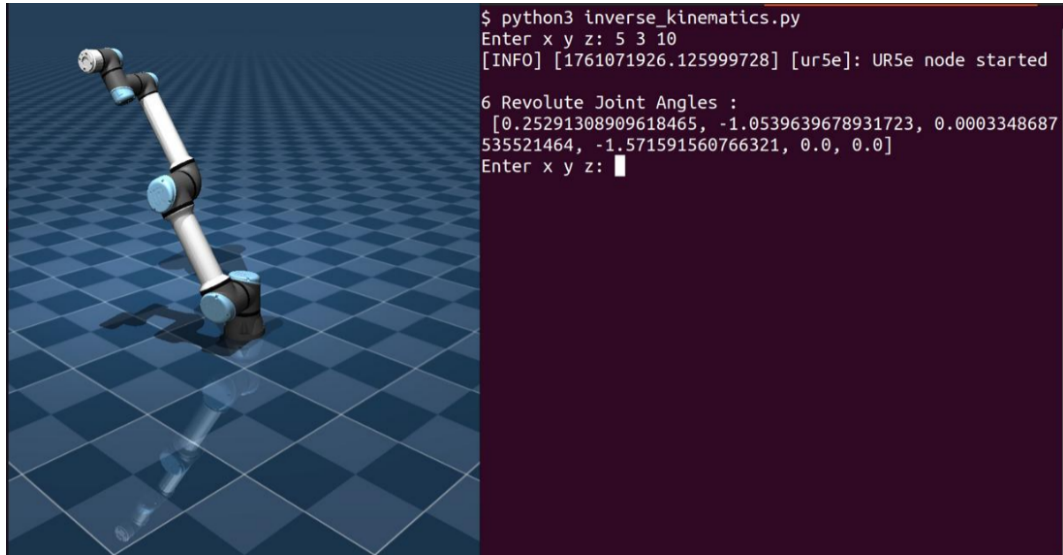


Figure 5: inverse kinematics

3 Velocity and Acceleration Kinematics

Here we calculate forward and inverse kinematics for both velocity and acceleration

3.1 Forward Velocity Kinematics

For this section, the task was to get the kinematics for both the velocity and acceleration. Starting with the forward velocity kinematics, the goal is to map the joint velocities in the joint space to the end-effector velocities in the task space. To achieve this, the previously derived equations for the end-effector position, expressed in terms of the joint variables q , were used. The closed form equations for the end-effector position can be seen in Table 1.

Velocity is defined as the rate of change of position with respect to time. In other words, velocity is derived by differentiating the position with respect to time. However, since the position components are not functions of time, we use the chain rule:

$$\frac{dx}{dt} = \frac{dx}{dq} \cdot \frac{dq}{dt}$$

Since a robot arm has multiple joint variables q , the velocity of the end-effector's component can be expressed as partial derivatives. Each partial derivative is the velocity of a specific joint for a position component. Applying this to the UR5E robot arm, which has 6 joint variables $q_1, q_2, q_3, q_4, q_5, q_6$, results in the following end-effector's x-component velocity:

$$\frac{dx}{dt} = \frac{\partial x}{\partial q_1} \cdot \frac{dq_1}{dt} + \frac{\partial x}{\partial q_2} \cdot \frac{dq_2}{dt} + \frac{\partial x}{\partial q_3} \cdot \frac{dq_3}{dt} + \frac{\partial x}{\partial q_4} \cdot \frac{dq_4}{dt} + \frac{\partial x}{\partial q_5} \cdot \frac{dq_5}{dt} + \frac{\partial x}{\partial q_6} \cdot \frac{dq_6}{dt}$$

This relationship can be compactly represented in matrix form as:

$$\dot{X} = J \cdot \dot{q}$$

Here, $\dot{X}$ is the end-effector velocities, $\dot{q}$ is the joint velocities, and J is the Jacobian matrix, which contains the partial derivatives of the end-effector position with respect to each joint variable. It basically maps the joint velocities to the end-effector velocities. For the UR5E robot arm, expanding these matrices gives:

$$\begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix} = \begin{bmatrix} J_{1,1} & J_{1,2} & J_{1,3} & J_{1,4} & J_{1,5} & J_{1,6} \\ J_{2,1} & J_{2,2} & J_{2,3} & J_{2,4} & J_{2,5} & J_{2,6} \\ J_{3,1} & J_{3,2} & J_{3,3} & J_{3,4} & J_{3,5} & J_{3,6} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \\ \dot{q}_4 \\ \dot{q}_5 \\ \dot{q}_6 \end{bmatrix}$$

Using the python library SymPy, each $J_{i,j}$ was computed, with the resulting partial derivatives shown in Table 2.

Jacobian Element	Partial Derivative (Equation)
$J_{1,1} = \frac{\partial x}{\partial q_1}$	$0.1 \sin(q_1) \sin(q_5) \cos(q_2 + q_3 + q_4) - 0.1 \sin(q_1) \sin(q_2 + q_3 + q_4) + 0.425 \sin(q_1) \cos(q_2) + 0.392 \sin(q_1) \cos(q_2 + q_3) + 0.1 \cos(q_1) \cos(q_5) + 0.127 \cos(q_1)$
$J_{1,2} = \frac{\partial x}{\partial q_2}$	$0.425 \sin(q_2) \cos(q_1) + 0.1 \sin(q_5) \sin(q_2 + q_3 + q_4) \cos(q_1) + 0.392 \sin(q_2 + q_3) \cos(q_1) + 0.1 \cos(q_1) \cos(q_2 + q_3 + q_4)$
$J_{1,3} = \frac{\partial x}{\partial q_3}$	$0.1 \sin(q_5) \sin(q_2 + q_3 + q_4) \cos(q_1) + 0.392 \sin(q_2 + q_3) \cos(q_1) + 0.1 \cos(q_1) \cos(q_2 + q_3 + q_4)$
$J_{1,4} = \frac{\partial x}{\partial q_4}$	$0.1 \sin(q_5) \sin(q_2 + q_3 + q_4) \cos(q_1) + 0.1 \cos(q_1) \cos(q_2 + q_3 + q_4)$
$J_{1,5} = \frac{\partial x}{\partial q_5}$	$-0.1 \sin(q_1) \sin(q_5) - 0.1 \cos(q_1) \cos(q_5) \cos(q_2 + q_3 + q_4)$
$J_{1,6} = \frac{\partial x}{\partial q_6}$	0
$J_{2,1} = \frac{\partial y}{\partial q_1}$	$0.1 \sin(q_1) \cos(q_5) + 0.127 \sin(q_1) - 0.1 \sin(q_5) \cos(q_1) \cos(q_2 + q_3 + q_4) + 0.1 \sin(q_2 + q_3 + q_4) \cos(q_1) - 0.425 \cos(q_1) \cos(q_2) - 0.392 \cos(q_1) \cos(q_2 + q_3)$
$J_{2,2} = \frac{\partial y}{\partial q_2}$	$0.425 \sin(q_1) \sin(q_2) + 0.1 \sin(q_1) \sin(q_5) \sin(q_2 + q_3 + q_4) + 0.392 \sin(q_1) \sin(q_2 + q_3) + 0.1 \sin(q_1) \cos(q_2 + q_3 + q_4)$
$J_{2,3} = \frac{\partial y}{\partial q_3}$	$0.1 \sin(q_1) \sin(q_5) \sin(q_2 + q_3 + q_4) + 0.392 \sin(q_1) \sin(q_2 + q_3) + 0.1 \sin(q_1) \cos(q_2 + q_3 + q_4)$
$J_{2,4} = \frac{\partial y}{\partial q_4}$	$0.1 \sin(q_1) \sin(q_5) \sin(q_2 + q_3 + q_4) + 0.1 \sin(q_1) \cos(q_2 + q_3 + q_4)$
$J_{2,5} = \frac{\partial y}{\partial q_5}$	$-0.1 \sin(q_1) \cos(q_5) \cos(q_2 + q_3 + q_4) + 0.1 \sin(q_5) \cos(q_1)$
$J_{2,6} = \frac{\partial y}{\partial q_6}$	0
$J_{3,1} = \frac{\partial z}{\partial q_1}$	0
$J_{3,2} = \frac{\partial z}{\partial q_2}$	$-0.1 \sin(q_5) \cos(q_2 + q_3 + q_4) + 0.1 \sin(q_2 + q_3 + q_4) - 0.425 \cos(q_2) - 0.392 \cos(q_2 + q_3)$
$J_{3,3} = \frac{\partial z}{\partial q_3}$	$-0.1 \sin(q_5) \cos(q_2 + q_3 + q_4) + 0.1 \sin(q_2 + q_3 + q_4) - 0.392 \cos(q_2 + q_3)$
$J_{3,4} = \frac{\partial z}{\partial q_4}$	$-0.1 \sin(q_5) \cos(q_2 + q_3 + q_4) + 0.1 \sin(q_2 + q_3 + q_4)$
$J_{3,5} = \frac{\partial z}{\partial q_5}$	$-0.1 \sin(q_2 + q_3 + q_4) \cos(q_5)$
$J_{3,6} = \frac{\partial z}{\partial q_6}$	0

Table 2: Jacobian elements $J_{i,j}$ and their corresponding partial derivatives

3.2 Inverse Velocity Kinematics

The goal of inverse velocity kinematics is essentially the reverse of forward velocity kinematics. Instead of mapping joint velocities to end-effector velocities, it maps the end-effector velocities in task space to the corresponding joint velocities in joint space. This can be achieved by applying the inverse of the Jacobian matrix to both sides of the forward velocity equation:

$$\begin{aligned}\dot{X} &= J \cdot \dot{q} \\ \dot{q} &= J^{-1} \cdot \dot{X}\end{aligned}$$

However, since the Jacobian matrix derived for the UR5E robot arm is not a square matrix, the pseudo-inverse J^+ is used instead, resulting in:

$$\dot{q} = J^+ \cdot \dot{X} = J^T (J J^T)^{-1} \cdot \dot{X}$$

Expanding the matrices results in the following:

$$\begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \\ \dot{q}_4 \\ \dot{q}_5 \\ \dot{q}_6 \end{bmatrix} = \begin{bmatrix} J_{1,1}^+ & J_{1,2}^+ & J_{1,3}^+ \\ J_{2,1}^+ & J_{2,2}^+ & J_{2,3}^+ \\ J_{3,1}^+ & J_{3,2}^+ & J_{3,3}^+ \\ J_{4,1}^+ & J_{4,2}^+ & J_{4,3}^+ \\ J_{5,1}^+ & J_{5,2}^+ & J_{5,3}^+ \\ J_{6,1}^+ & J_{6,2}^+ & J_{6,3}^+ \end{bmatrix} \begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix}$$

Here, the Jacobian matrix shown is after applying the pseudo-inverse. Due to the intense computation of deriving explicit expressions for each $J_{i,j}^+$, their values are not shown and instead computed numerically once the joint values are known.

3.3 Forward Acceleration Kinematics

The forward acceleration kinematics builds on the forward velocity kinematics. To derive it, the forward velocity is differentiated with respect to time. Similarly to before, the chain rule is used. The product rule is also applied since they are two separate functions, giving us:

$$\ddot{X} = \frac{d}{dt} \dot{X} = \frac{d}{dt} (J \dot{q})$$

$$\ddot{X} = \dot{J} \dot{q} + J \ddot{q}$$

Here, $\ddot{X}$ is the end-effector acceleration, $\ddot{q}$ is the joint accelerations, $\dot{q}$ is the joint velocities, J is the Jacobian matrix, and $\dot{J}$ is its time derivative. This expression maps the joints accelerations to the end-effector acceleration. Expanding these matrices for the UR5E robot arm once again results in:

$$\begin{bmatrix} \ddot{X} \\ \ddot{Y} \\ \ddot{Z} \end{bmatrix} = \begin{bmatrix} \dot{J}_{1,1} & \dot{J}_{1,2} & \dot{J}_{1,3} & \dot{J}_{1,4} & \dot{J}_{1,5} & \dot{J}_{1,6} \\ \dot{J}_{2,1} & \dot{J}_{2,2} & \dot{J}_{2,3} & \dot{J}_{2,4} & \dot{J}_{2,5} & \dot{J}_{2,6} \\ \dot{J}_{3,1} & \dot{J}_{3,2} & \dot{J}_{3,3} & \dot{J}_{3,4} & \dot{J}_{3,5} & \dot{J}_{3,6} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \\ \dot{q}_4 \\ \dot{q}_5 \\ \dot{q}_6 \end{bmatrix} + \begin{bmatrix} J_{1,1} & J_{1,2} & J_{1,3} & J_{1,4} & J_{1,5} & J_{1,6} \\ J_{2,1} & J_{2,2} & J_{2,3} & J_{2,4} & J_{2,5} & J_{2,6} \\ J_{3,1} & J_{3,2} & J_{3,3} & J_{3,4} & J_{3,5} & J_{3,6} \end{bmatrix} \begin{bmatrix} \ddot{q}_1 \\ \ddot{q}_2 \\ \ddot{q}_3 \\ \ddot{q}_4 \\ \ddot{q}_5 \\ \ddot{q}_6 \end{bmatrix}$$

The values of J can be seen in Table 2. Due to the length of each element $\dot{J}_{i,j}$, it is not shown in the report, however it can be seen by running the code `jacobian_dot_matrix.py` if needed.

3.4 Inverse Acceleration Kinematics

Inverse acceleration kinematics does the opposite of forward acceleration kinematics, where it maps the end-effector's acceleration to the joints accelerations. Similar to inverse velocity kinematics, this can be obtained through simple algebraic manipulation.

$$\ddot{X} = \dot{J}\dot{q} + J\ddot{q}$$

$$J\ddot{q} = \ddot{X} - \dot{J}\dot{q}$$

$$\boxed{\ddot{q} = J^+ \left(\ddot{X} - \dot{J}\dot{q} \right)}$$

As mentioned before, J^+ is the psuedo-inverse of J due to it not being a square matrix. Expanding these matrices gives us:

$$\begin{bmatrix} \ddot{q}_1 \\ \ddot{q}_2 \\ \ddot{q}_3 \\ \ddot{q}_4 \\ \ddot{q}_5 \\ \ddot{q}_6 \end{bmatrix} = \begin{bmatrix} J_{1,1}^+ & J_{1,2}^+ & J_{1,3}^+ \\ J_{2,1}^+ & J_{2,2}^+ & J_{2,3}^+ \\ J_{3,1}^+ & J_{3,2}^+ & J_{3,3}^+ \\ J_{4,1}^+ & J_{4,2}^+ & J_{4,3}^+ \\ J_{5,1}^+ & J_{5,2}^+ & J_{5,3}^+ \\ J_{6,1}^+ & J_{6,2}^+ & J_{6,3}^+ \end{bmatrix} \left(\begin{bmatrix} \ddot{X} \\ \ddot{Y} \\ \ddot{Z} \end{bmatrix} - \begin{bmatrix} \dot{J}_{1,1} & \dot{J}_{1,2} & \dot{J}_{1,3} & \dot{J}_{1,4} & \dot{J}_{1,5} & \dot{J}_{1,6} \\ \dot{J}_{2,1} & \dot{J}_{2,2} & \dot{J}_{2,3} & \dot{J}_{2,4} & \dot{J}_{2,5} & \dot{J}_{2,6} \\ \dot{J}_{3,1} & \dot{J}_{3,2} & \dot{J}_{3,3} & \dot{J}_{3,4} & \dot{J}_{3,5} & \dot{J}_{3,6} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \\ \dot{q}_4 \\ \dot{q}_5 \\ \dot{q}_6 \end{bmatrix} \right)$$

All derived equations were computed using NumPy and SymPy built-in libraries.

4 Trajectory Planning

4.1 Joint-Space Trajectory

We calculated the joint space trajectory using the following equation:

$$q(t) = C_0 + C_1t + C_2t^2 + C_3t^3$$

$$\dot{q}(t) = C_1 + C_2t + C_3t^2$$

where $C_0 = q_{\text{initial}}$, $C_1 = [0,0,0,0,0,0]$ due to initial velocity being 0, and C_2 and C_3 are calculated from solving q and $\dot{q}$ simultaneously with q initial and final being:

$$q_{\text{initial}} = (-2.78, -0.58, 1.10, -2.09, 1.57, 0.0)$$

$$q_{\text{final}} = (-4.48, -0.91, 0.91, -1.57, 1.57, 0.0)$$

The test trajectory goes from (0.7, 0.4, 0.3) to (0.3, -0.7, 0.6) in 10 seconds. Since it is joint space trajectory, the motion corresponds to moving the robot from the initial joint configuration q_{initial} to the final joint configuration q_{final} . A timestep of 0.001 seconds is used due to the simulation frequency itself being 1000 HZ, so 1/1000 is 0.001 seconds.

4.2 Task-Space Trajectory

The arm travels in a straight line, with a new point being calculated and sent to the robot arm every timestep (0.001 seconds)

Since the robot arm is traveling in a straight line, simple straight line equation are used to calculate x,y,z values and are defined as:

$$x(t) = x_{initial} + alpha * t$$

applying to x,y,z and calculating alpha from the final position of gives :

$$x(t) = 0.7 - 0.04t$$

$$y(t) = 0.4 - 0.11t$$

$$z(t) = 0.3 + 0.03t$$

These equations are then used to find the position of the end effector at each timestep,

5 Fuzzy Logic Controller & Closed Loop Trajectory

Here we make our robot follow a closed loop trajectory and use fuzzy logic to keep it on said trajecotry.

5.1 Fuzzy Logic Controller (FLC)

To ensure the robot follows its trajectory accurately, a fuzzy logic controller (FLC) is used to continuously correct the position error. The controller reduces the tracking error by applying a velocity command that opposes the direction of error of the actual value from expected value where each input error(e) has a corresponding output velocity(v).

The FLC uses the following fuzzy sets for the error input (e):

LN → Large Negative error

MN → Medium Negative error

S → Small (near zero) error

MP → Medium Positive error

LP → Large Positive error

The same fuzzy sets are used for the output velocity (v):

LN → Large Negative velocity

MN → Medium Negative velocity

S → Small / Zero velocity

MP → Medium Positive velocity

LP → Large Positive velocity

Set	a (start)	b (peak)	c (end)
NL	-2π	$-\frac{3\pi}{4}$	$-\frac{\pi}{2}$
NS	$-\frac{2\pi}{3}$	$-\frac{\pi}{2}$	0
Z	-0.1	0	0.1
PS	0	$\frac{\pi}{2}$	$\frac{2\pi}{3}$
PL	$\frac{\pi}{2}$	$\frac{3\pi}{4}$	2π

Table 3: Fuzzy sets for **position error** e .

Set	a (start)	b (peak)	c (end)
NF	$-\pi$	$-\frac{3\pi}{4}$	$-\frac{\pi}{2}$
NS	$-\pi$	$-\frac{\pi}{2}$	0
Z	-0.1	0	0.1
PS	0	$\frac{\pi}{2}$	π
PF	$\frac{\pi}{2}$	$\frac{3\pi}{4}$	π

Table 4: Fuzzy sets for **velocity** v .

The membership functions, which determine how strongly a numeric error belongs to each fuzzy set, are defined as follows:

$$\begin{aligned}
\mu_{LN}(e) &= \max\left(0, \min\left(1, \frac{-2-e}{2}\right)\right) && \text{(Large Negative)} \\
\mu_{MN}(e) &= \max(0, 1 - |e + 1|) && \text{(Medium Negative)} \\
\mu_S(e) &= \max(0, 1 - |e|) && \text{(Small / Near Zero)} \\
\mu_{MP}(e) &= \max(0, 1 - |e - 1|) && \text{(Medium Positive)} \\
\mu_{LP}(e) &= \max\left(0, \min\left(1, \frac{e-2}{2}\right)\right) && \text{(Large Positive)}
\end{aligned}$$

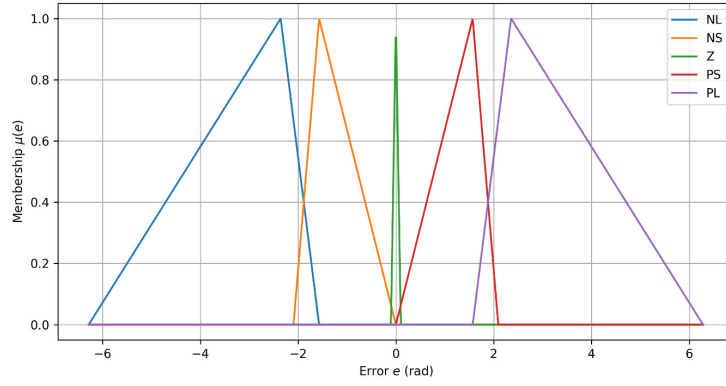


Figure 6: Membership functions for error fuzzy sets

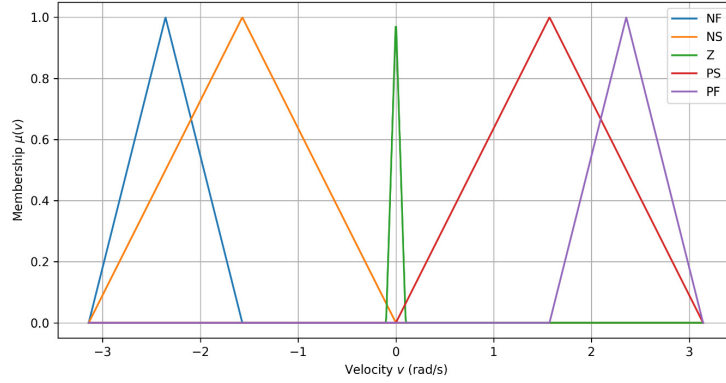


Figure 7: Membership functions for velocity fuzzy sets

These membership functions convert the numeric error into a degree of belonging for each fuzzy set, which is then used with the fuzzy rule table to calculate the output velocity via defuzzification.

5.2 Closed Loop Trajectory

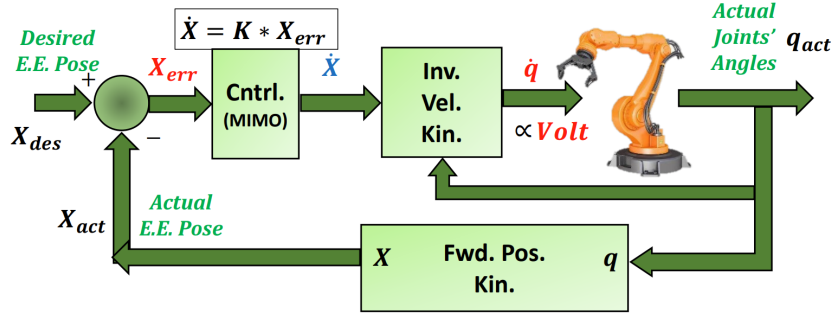


Figure 8: Closed-loop fuzzy-logic-based control scheme.

As shown in Figure 8, the proposed closed-loop control scheme generates a sequence of trajectory points in Cartesian space and sends them to the fuzzy logic controller (FLC), which computes the desired end-effector linear and angular velocities required to follow the reference path. These end-effector velocities are then mapped into joint-space velocities by the inverse velocity kinematics, using the manipulator Jacobian to obtain the corresponding joint rates that are applied as commands to the robot actuators. The resulting joint motions are used in the forward kinematics to compute the actual end-effector pose, which is compared with the expected trajectory point to obtain a position error that is fed back to update the input of the FLC and generate new corrected trajectory points, thereby closing the loop and improving tracking accuracy.

Final Result

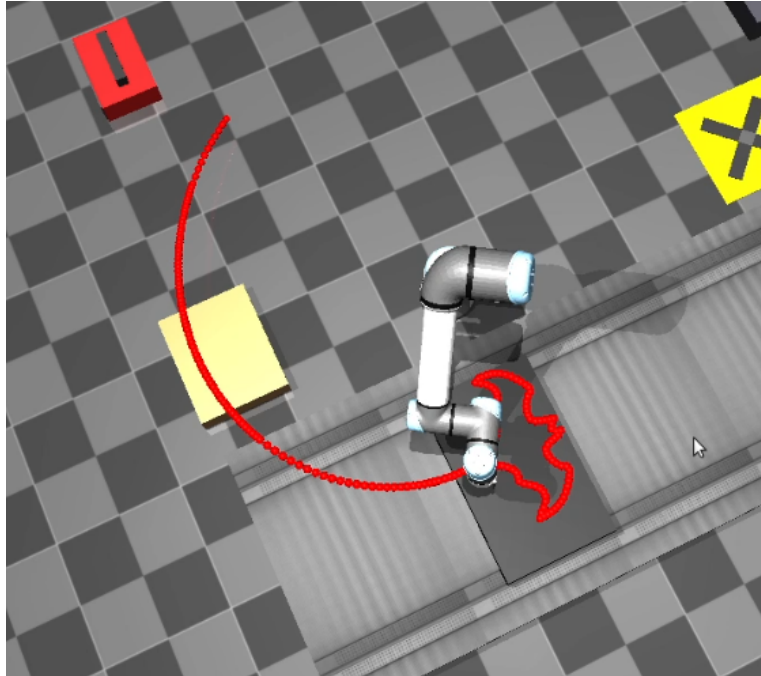


Figure 9: Robot Laser Cutting Batman Logo

References

- [1] The Tech Museum of Innovation. Robotics: About the exhibition, 2008. Retrieved 2008-09-15 from The Tech Museum of Innovation.
- [2] E. Roberts. Robotics: A brief history.
- [3] M. E. Moran. Evolution of robotic arms. *Journal of Robotic Surgery*, 2007.
- [4] Augmentus. The history of robotic arms, 2025. Retrieved from <https://www.augmentus.tech/blog/the-history-of-robotic-arms/>.
- [5] A. L. G. Morrell. The history of robotic surgery and its evolution. *PMC*, 2021.
- [6] EurekaSelect. A large model-based embodied intelligent robotic arm system, 2025. Retrieved from <https://www.eurekaselect.com/245245/article>.
- [7] Rahul Gautam, Ankush Gedam, Ashish Zade, and Ajay Mahawadiwar. Review on development of industrial robotic arm. *Abha Gaikwad-Patil College of Engineering, Nagpur, Maharashtra, India*, 2020. Department of Mechanical Engineering.
- [8] Mangesh Dindayal Nikose, Omkar S. Vaidya, and Swapnil Bandu Ambekar. Future of robotic laser cutting in the industry 4.0 era. *International Journal of Emerging Trends in Engineering Research*, 2024.
- [9] Universal Robots A/S. Ur5e, 2025.